

เอกสารข้อกำหนดและการออกแบบระบบ (System Requirements and Design Specification)

Project Name: Home & Appliance Maintenance Log

Deployment: Static Frontend via GitHub Pages

Tech Stack: React (Vite) + Tailwind CSS + shadcn/ui

Database: Local Client-Side Database (SQLite WASM)

1. Business Requirement Document (BRD)

1.1 วัตถุประสงค์ของโครงการ (Project Objective)

เพื่อพัฒนาระบบบริหารจัดการและติดตามประวัติการซ่อมบำรุงสินทรัพย์ภายในบ้านแบบส่วนตัว ช่วยยืดอายุการใช้งาน ป้องกันการชำรุดก่อนเวลาอันควร และอำนวยความสะดวกในการวางแผนรอบการบำรุงรักษา รวมถึงการติดตามค่าใช้จ่ายที่เกี่ยวข้อง

1.2 ขอบเขตของระบบ (System Scope)

In-Scope:

- รองรับการใช้งานแบบ Single-user บนอุปกรณ์มือถือและแท็บเล็ตผ่านเว็บเบราว์เซอร์
- ข้อมูลทั้งหมดจัดเก็บอยู่บนอุปกรณ์ของผู้ใช้ (Client-side) ด้วยหลักการ Offline-First
- การบันทึกข้อมูลอุปกรณ์ ประวัติการซ่อมบำรุง และระบบประมวลผลการแจ้งเตือน (UI Alerts)
- ฟังก์ชันนำเข้าและส่งออกข้อมูล (Import/Export) เพื่อสำรองข้อมูล

Out-of-Scope:

- ไม่มีการเชื่อมต่อฐานข้อมูลส่วนกลาง (No Cloud Backend)
- ไม่รองรับระบบผู้ใช้งานหลายคน (No Multi-user/Authentication)
- ไม่มีการแจ้งเตือนแบบ Push Notification ผ่านระบบปฏิบัติการ (OS Level) ขณะปิดแอปพลิเคชัน

2. Functional Requirements (FR)

FR-01: Asset Management (ระบบจัดการข้อมูลเครื่องใช้ไฟฟ้า/สินทรัพย์)

- **FR-01.1:** ผู้ใช้สามารถ เพิ่ม แก้ไข ลบ ข้อมูลเครื่องใช้ไฟฟ้า (เช่น ยี่ห้อ, รุ่น, วันที่ซื้อ, ระยะเวลาประกัน, สถานที่ติดตั้ง)
- **FR-01.2:** สามารถแนบรูปภาพอ้างอิงของอุปกรณ์หรือใบเสร็จได้ (โดยระบบจะทำการย่อขนาดและแปลงเป็น Base64 String เพื่อจัดเก็บใน Local Database)

FR-02: Task Template Management (ระบบจัดการรูปแบบการบำรุงรักษา)

- **FR-02.1:** ผู้ใช้สามารถสร้างรูปแบบงานมาตรฐาน (Task) ที่ผูกกับอุปกรณ์แต่ละชิ้นได้ เช่น "ล้างแผ่นกรองแอร์", "เปลี่ยนไส้กรองน้ำดื่ม"
- **FR-02.2:** สามารถกำหนดรอบความถี่ของการบำรุงรักษาได้ (ระบุเป็นจำนวนวัน เช่น 30 วัน, 180 วัน, 365 วัน)

FR-03: Maintenance Log (ระบบบันทึกประวัติการดำเนินการ)

- **FR-03.1:** ผู้ใช้สามารถบันทึกประวัติการทำงานจริงที่อ้างอิงจาก Task Template ได้
- **FR-03.2:** สามารถระบุวันที่ดำเนินการเสร็จสิ้น ค่าใช้จ่ายที่เกิดขึ้น และบันทึกข้อความหมายเหตุ (Remarks) ได้

FR-04: Dashboard & Rule-based Notification (ระบบสรุปผลและการแจ้งเตือน)

- **FR-04.1:** ระบบประมวลผลหาวันครบกำหนดครั้งถัดไป (Next Due Date) อัตโนมัติ โดยคำนวณจาก `วันที่ดำเนินการล่าสุด + รอบความถี่ (วัน)`
- **FR-04.2:** หน้า Dashboard จะต้องแสดงรายการงานที่ "ถึงกำหนดแล้ว (Overdue)" และ "กำลังจะถึงกำหนด (Upcoming)" ในอีก 7-14 วันข้างหน้า
- **FR-04.3:** หน้า Dashboard แสดงสรุปค่าใช้จ่ายการซ่อมบำรุงรวม

FR-05: Data Persistence & Backup (ระบบจัดการไฟล์ข้อมูล)

- **FR-05.1:** ระบบมีฟังก์ชัน "Export Data" เพื่อสร้างไฟล์ Backup (.json หรือ .sqlite) ลงในเครื่องของผู้ใช้
- **FR-05.2:** ระบบมีฟังก์ชัน "Import Data" เพื่อกู้คืนข้อมูลหรือย้ายอุปกรณ์ใช้งาน

3. Non-Functional Requirements (NFR)

- **NFR-01: Availability & Offline Capability:** ระบบต้องเป็น Progressive Web App (PWA) โหลดใช้งานได้สมบูรณ์แบบ 100% แม้ไม่มีสัญญาณอินเทอร์เน็ต (Offline mode) ผ่านเทคโนโลยี Service Worker
- **NFR-02: Performance:** ระยะเวลาในการ Query ข้อมูลและคำนวณ Dashboard ต้องน้อยกว่า 1 วินาที โดยใช้ Vite เป็น Build Tool เพื่อจัดการ Asset size ให้มีขนาดเล็กและโหลดไวที่สุด

- **NFR-03: Data Privacy & Security:** ข้อมูลทั้งหมดต้องถูกประมวลผลและกักเก็บอยู่ใน Local Storage/IndexedDB ของเบราว์เซอร์ผู้ใช้เท่านั้น ห้ามส่งข้อมูลใดๆ ออกไปยังเซิร์ฟเวอร์ภายนอก
- **NFR-04: Portability & Deployment:** แอปพลิเคชันต้องสามารถ Build เป็น Static Files และทำ Automated Deployment ขึ้น GitHub Pages ผ่านไลบรารี `gh-pages` ได้โดยตรง
- **NFR-05: Usability & UI/UX:** ส่วนติดต่อผู้ใช้งานต้องมีความทันสมัย เป็น Responsive Design (Mobile-first) โดยใช้ **Tailwind CSS** ในการจัดการสไตล์ และใช้ **shadcn/ui** สำหรับโครงสร้าง Component ที่รองรับ Accessibility (a11y) มาตรฐานสูง

4. System Design Specification (SDS)

4.1 System Architecture

- **Frontend Framework:** React 18+ (ตั้งค่าโปรเจกต์และ Build ด้วย Vite)
- **UI/UX Components:** shadcn/ui (Radix UI primitives)
- **Styling:** Tailwind CSS (Utility-first CSS framework)
- **Database Engine:** SQLite WASM (ประมวลผล SQL ผ่าน WebAssembly ในเบราว์เซอร์)
- **Storage Layer:** ข้อมูล Database file ถูกเขียนและอ่านผ่าน IndexedDB เพื่อเลี่ยงข้อจำกัด OPFS บน GitHub Pages
- **Hosting & Deployment:** GitHub Pages (ผ่านสคริปต์ `npm run deploy` ที่ใช้แพ็คเกจ `gh-pages`)

4.2 Database Schema (Data Model)

ใช้หลักการออกแบบฐานข้อมูลเชิงสัมพันธ์ (Relational Model) เพื่อให้รองรับการ Query ข้อมูลที่ซับซ้อนได้อย่างมีประสิทธิภาพ

Table 1: APPLIANCE (ข้อมูลสินทรัพย์)

Field Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto Increment	รหัสอ้างอิงอุปกรณ์
name	TEXT	Not Null	ชื่อเรียก (เช่น แอร์ห้องนอน)
brand	TEXT	Nullable	ยี่ห้อ
model_number	TEXT	Nullable	รุ่น / ซีเรียลนัมเบอร์
location	TEXT	Nullable	จุดติดตั้ง (เช่น ห้องครัว)
purchase_date	TEXT	Nullable	วันที่ซื้อ (จัดเก็บแบบ ISO 8601 YYYY-MM-DD)
warranty_months	INTEGER	Nullable	ระยะเวลาประกัน (เดือน)
image_data	TEXT	Nullable	รูปภาพ Base64 format

Table 2: MAINTENANCE_TASK (แผนการบำรุงรักษา)

Field Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto Increment	รหัสอ้างอิงแผนงาน
appliance_id	INTEGER	FK (APPLIANCE.id)	อ้างอิงอุปกรณ์
task_name	TEXT	Not Null	ชื่อรายการ (เช่น ถอดล้าง)
frequency_days	INTEGER	Not Null	รอบระยะเวลา (วัน)
is_active	BOOLEAN	Default 1	สถานะเปิด/ปิดการแจ้งเตือน

Table 3: MAINTENANCE_LOG (ประวัติการดำเนินการจริง)

Field Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto Increment	รหัสอ้างอิงประวัติ
task_id	INTEGER	FK (MAINTENANCE_TASK.id)	อ้างอิงงานบำรุงรักษา
performed_date	TEXT	Not Null	วันที่ดำเนินการ (จัดเก็บแบบ ISO 8601 YYYY-MM-DD)
cost	DECIMAL	Default 0.00	ค่าใช้จ่ายที่เกิดขึ้น
remarks	TEXT	Nullable	หมายเหตุเพิ่มเติม

4.3 Core Algorithms (ลอจิกสำคัญ)

การประมวลผล Dashboard แจ้งเตือน:

ในการแสดงผลหน้าแรก ระบบจะรัน SQL Query เพื่อดึงรายการงาน (Task) ที่ใช้งานอยู่ ($is_active = 1$) ไป LEFT JOIN กับ MAINTENANCE_LOG ที่มี performed_date ล่าสุด จากนั้นนำ performed_date + frequency_days มาเปรียบเทียบกับวันที่ปัจจุบัน (Current Date)

- หาก $Next\ Due\ Date \leq Current\ Date$ → จัดเข้ากลุ่ม "Overdue (เลยกำหนด)" (แสดงผลด้วย UI สีแดงผ่าน Tailwind utility class text-red-500 bg-red-50)
- หาก $Next\ Due\ Date \leq Current\ Date + 14$ → จัดเข้ากลุ่ม "Upcoming (ใกล้ถึงกำหนด)" (แสดงผลด้วย UI สีเหลืองผ่าน Tailwind utility class text-amber-500 bg-amber-50)

5. Technical Mitigation Strategies (การจัดการข้อจำกัดเชิงเทคนิค)

เพื่อแก้ไขข้อจำกัดด้านสถาปัตยกรรม Static Hosting และ Local Storage ระบบได้กำหนดมาตรการเชิงเทคนิค (Stable Solutions) ดังต่อไปนี้:

5.1 การจัดการพื้นที่จัดเก็บ SQLite WASM บน GitHub Pages (Storage Strategy)

ปัญหา: GitHub Pages ไม่รองรับการกำหนด HTTP Headers (COOP/COEP) ซึ่งจำเป็นสำหรับการรับ OPFS (Origin Private File System) เพื่อให้ SQLite WASM ทำงานได้เต็มประสิทธิภาพ

วิธีจัดการ (Stable Solution):

- **ใช้ IndexedDB VFS (Virtual File System):** กำหนดให้ SQLite WASM เขียนข้อมูลลง IndexedDB แทน OPFS แม้ประสิทธิภาพการเขียนจะช้ากว่า OPFS เล็กน้อย แต่รับประกันความเสถียร 100% ว่าสามารถรันบน GitHub Pages ได้โดยไม่ต้องพึ่งพา Service Worker Hack
- **ใช้ LocalForage หรือ Dexie.js (ทางเลือกสำรอง):** หากพบปัญหาในการคอมไพล์ SQLite WASM จะปรับแผนไปใช้ NoSQL database wrap (เช่น Dexie.js) ในการจัดการ IndexedDB แทน

5.2 การป้องกันข้อมูลสูญหายจาก Browser Eviction (Data Persistence)

ปัญหา: ข้อมูลใน IndexedDB/Local Storage อาจถูกระบบปฏิบัติการลบทิ้งหากพื้นที่มือถือเต็ม หรือผู้ใช้ล้างแคชเบราว์เซอร์

วิธีจัดการ (Stable Solution):

- **ใช้ File System Access API:** สำหรับเบราว์เซอร์ที่รองรับ (เช่น Chrome/Edge บน Desktop) ผู้ใช้สามารถตั้งค่าให้ระบบ "Save" ไฟล์ฐานข้อมูล `.sqlite` ลงในไดเรกทอรีส่วนตัว (เช่น Documents) ได้โดยตรง
- **Automated Local Backup Prompt:** สร้างระบบ UI Prompt เต็มเตือนให้ผู้ใช้กดปุ่ม Download ไฟล์ Backup (Export Data) เป็นประจำทุก 30 วัน หากระบบตรวจพบว่าไม่มีการ Export มาเกินกำหนด

5.3 การควบคุมขนาดฐานข้อมูล (Image Bloat Management)

ปัญหา: การจัดเก็บรูปภาพในรูปแบบ Base64 String ลงในฐานข้อมูลจะกินพื้นที่มาก และอาจทำให้ IndexedDB ทำงานช้าเมื่อมีจำนวนภาพเพิ่มขึ้น

วิธีจัดการ (Stable Solution):

- **Image Compression Pipeline:** ก่อนนำภาพบันทึกลง Database ภาพทุกรูปต้องผ่านไลบรารีบีบอัดฝั่ง Client (เช่น `browser-image-compression`)
- **Standardization:** บังคับลดขนาดความละเอียดภาพ (Resize) ให้มีความกว้างสูงสุดไม่เกิน 800 พิกเซล และคุณภาพระดับ WebP/JPEG ที่ 0.7 (เป้าหมายขนาดไฟล์ ไม่เกิน 200KB ต่อรูป)

5.4 การจัดการ React Routing บน Static Host (Routing Configuration)

ปัญหา: การใช้งาน React Router แบบดั้งเดิม (`BrowserRouter`) บน GitHub Pages เมื่อมีการรีเฟรชหน้าเว็บที่ URL ย่อย (เช่น `/add`) จะเกิดปัญหา 404 Not Found เนื่องจากไม่มีไฟล์ HTML มารองรับ

วิธีจัดการ (Stable Solution):

- **ใช้ HashRouter:** เปลี่ยนมาใช้ `HashRouter` จาก `react-router-dom` (เช่น `BrowserRouter`) เนื่องจาก Web Server จะสนใจเฉพาะ URL ส่วนที่อยู่หน้า `#` (ซึ่งคือ `index.html`) และปล่อยให้ React จัดการ Routing ส่วนที่อยู่หลัง `#` เอง วิธีนี้เสถียรที่สุด ไม่ต้องสร้างไฟล์ 404.html หรือเขียนสคริปต์ Redirect ให้ซับซ้อน

5.5 การจัดการชนิดข้อมูลวันที่ (Date & Thai Timezone Handling)

ปัญหา: SQLite ไม่รองรับ Data Type แบบ DATE/DATETIME โดยตรง การบันทึก Object Date ของ JavaScript อาจทำให้เกิดปัญหา Timezone Offset ทำให้ระบบแจ้งเตือนรอบบำรุงรักษาทำงานผิดพลาดเมื่อเปิดบนอุปกรณ์ที่ตั้งค่า Timezone ต่างกัน

วิธีจัดการ (Stable Solution):

- **Single Timezone Enforcement:** บังคับให้ระบบทั้งหมดทำงานภายใต้ Timezone ของประเทศไทย (`Asia/Bangkok` หรือ `UTC+07:00`) เพียงอย่างเดียว (Hardcoded Timezone) โดยไม่สนใจ Timezone ปัจจุบันของอุปกรณ์ที่รันแอปพลิเคชัน
- **Standardize Data Format:** จัดเก็บไฟล์วันที่ในฐานข้อมูลด้วยรูปแบบ Text ที่เป็น **ISO 8601 String** (`YYYY-MM-DD`) เสมอ
- **Timezone Independent Calculation:** ใช้ไลบรารีอย่าง `dayjs` พร้อมปลั๊กอิน `timezone` และ `utc` เพื่อประมวลผลวันที่และเวลาให้เป็น `Asia/Bangkok` ก่อนแสดงผลบน UI หรือคำนวณการบวกลบวัน (Frequency Days) เสมอ